



USER MANUAL

BEAMAGE series | Software Development Kit (SDK)

WARRANTY

First Year Warranty

The Gentec-EO Beamage series beam profilers carry a one-year warranty (from date of shipment) against material and/or workmanship defects, when used under normal operating conditions. The warranty does not cover damages related to battery leakage or any other misuse.

Any attempt by unauthorized personnel to alter or repair the product will void the warranty.

The manufacturer is not liable for consequential damages of any kind.

Contacting Gentec Electro-Optics Inc.

In case of malfunction, contact your local Gentec-EO distributor or nearest Gentec-EO Inc. office to obtain a return authorization number. The material should be returned to:

Gentec Electro-Optics, Inc.
445, St-Jean-Baptiste, Suite 160
Québec, QC
Canada, G2E 5N7

Tel: (418) 651-8003
Fax: (418) 651-1174
E-mail: service@gentec-eo.com

Website: gentec-eo.com

CLAIMS

To obtain warranty service, contact your nearest Gentec-EO agent or send the product, with a description of the problem, and prepaid transportation and insurance, to the nearest Gentec-EO agent. Gentec-EO Inc. assumes no risk for damage during transit. Gentec-EO Inc. will, at its discretion, repair or replace the defective product free of charge or refund your purchase price. However, if Gentec-EO Inc. determines that the failure is caused by misuse, alterations, accident, or improper conditions of operation or handling, the product shall therefore not be covered by the warranty.

TABLE OF CONTENTS

1. GENTEC-EO'S BEAMAGE SDK	5
1.1. WHAT IS THE <i>BEAMAGE SDK</i> ?	5
1.2. THE <i>.NET COMPATIBLE PIPELINE</i> FOR BEAMAGE CAMERAS	6
1.3. GETTING STARTED WITH THE <i>BEAMAGE SDK</i>	7
1.4. WHAT YOUR VISUAL STUDIO SOLUTION NEEDS	8
1.5. UNDERSTANDING THE CODE	8
1.6. <i>BEAMAGE SDK</i> CLASS DIAGRAM	9
2. QUICKSTART EXAMPLE	10
2.1. EXAMPLE: C# EXAMPLE WITH VIEWER	10
2.1.1. GENERAL INFORMATION	10
2.1.2. GRAB IMAGES	12
2.1.3. MATHEMATICAL OPERATIONS ON THE IMAGE BUFFER	15
2.1.4. EVENTS	16
3. BEAMAGE API	17
3.1. BCAM	17
<i>Class declaration</i>	17
<i>Properties</i>	17
<i>Events</i>	18
<i>Functions</i>	18
3.2. BCAMIMG	19
<i>Class declaration</i>	19
<i>Properties</i>	19
<i>Events</i>	20
<i>Functions</i>	20
3.3. BCAMSETTINGS	21
<i>Class declaration</i>	21
<i>Properties</i>	21
<i>Events</i>	21
3.4. BCAMPROPERTIES	22
<i>Class declaration</i>	22
<i>Properties</i>	22
<i>Events</i>	22
<i>Functions</i>	22
3.5. BERRORSMANAGER	23
<i>Class declaration</i>	23
<i>Properties</i>	23
<i>Events</i>	23
3.6. BSDK	24
<i>Class declaration</i>	24
<i>Properties</i>	24
<i>Events</i>	24
<i>Functions</i>	24

TABLE OF FIGURES

FIGURE 1 – CONCEPTUAL EXAMPLE OF <i>BEAMAGE SDK</i> INTEGRATION ON AN ASSEMBLY LINE.....	5
FIGURE 2 – A NEW WAY TO USE GENTEC-EO'S BEAMAGE CAMERA: THE <i>BEAMAGE SDK</i>	6
FIGURE 3 - FUNCTIONAL SCHEMATIC OF THE <i>.NET COMPATIBLE PIPELINE</i> FOR BEAMAGE CAMERAS.....	7
FIGURE 4 - <i>BEAMAGESDK.DLL</i> HAS BEEN ADDED AS A REFERENCE TO THE CURRENT <i>VISUAL STUDIO</i> PROJECT....	8
FIGURE 5 - <i>BEAMAGE SDK</i> CLASS DIAGRAM.....	9
FIGURE 6 - <i>BEAMAGE SDK SIMPLE VIEWER</i> USER INTERFACE WHEN A BEAMAGE-4M IS CONNECTED.....	10
FIGURE 7 - <i>BEAMAGE SDK SIMPLE VIEWER</i> USER INTERFACE WHEN TWO BEAMAGE-4.0 ARE CONNECTED.....	11
FIGURE 8 – SDK COLORS SCALE.....	20
FIGURE 9 - BEAMAGE IMAGE	20

1. Gentec-EO's *Beamage SDK*

1.1. What is the *Beamage SDK*?

Gentec-EO's *Beamage SDK* has been designed to help clients develop their own software user interface, make their own image analysis, and integrate the Beamage camera to their system without necessarily using Gentec-EO's *PC-BEAMAGE* software.

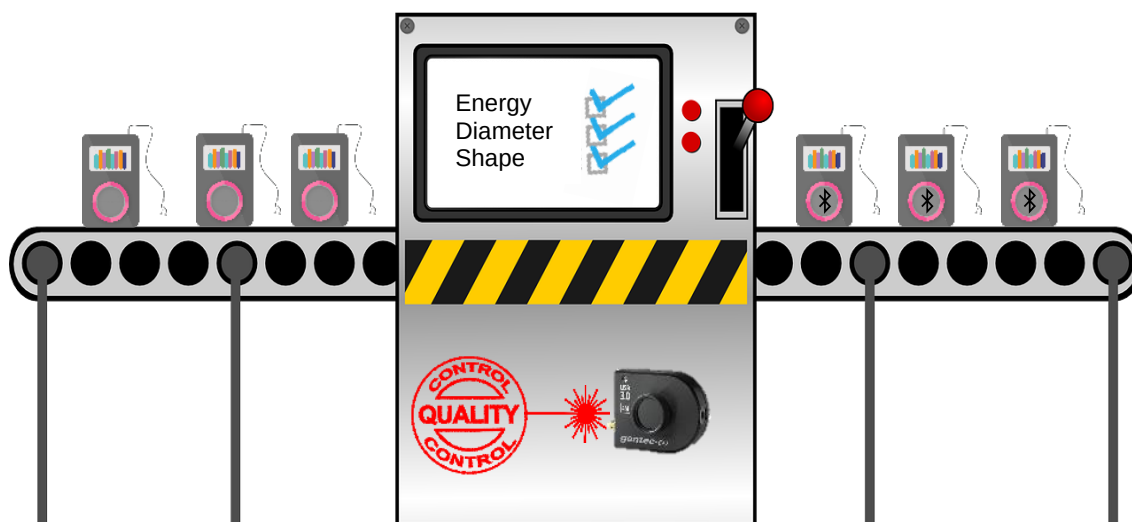


Figure 1 – Conceptual example of *Beamage SDK* integration on an assembly line

Gentec-EO's *Beamage SDK* is a *DLL* (Dynamic Link Library) that communicates with the camera drivers. It provides our customers with specific software functions to control the Beamage camera and retrieve images from it, in order to build custom Windows applications.

This *DLL* is written in C# and is compatible with all [.NET languages](#): C#, Visual Basic, F#, etc. Customers can build custom software or Windows applications to meet many specific needs: image analysis, hardware integration, quality control, assembly line safety, laboratory maintenance, etc.

Below is a simple schematic that illustrates the difference between 2 ways of managing the Beamage camera: the *Beamage SDK* and Gentec-EO's *PC-BEAMAGE* software.

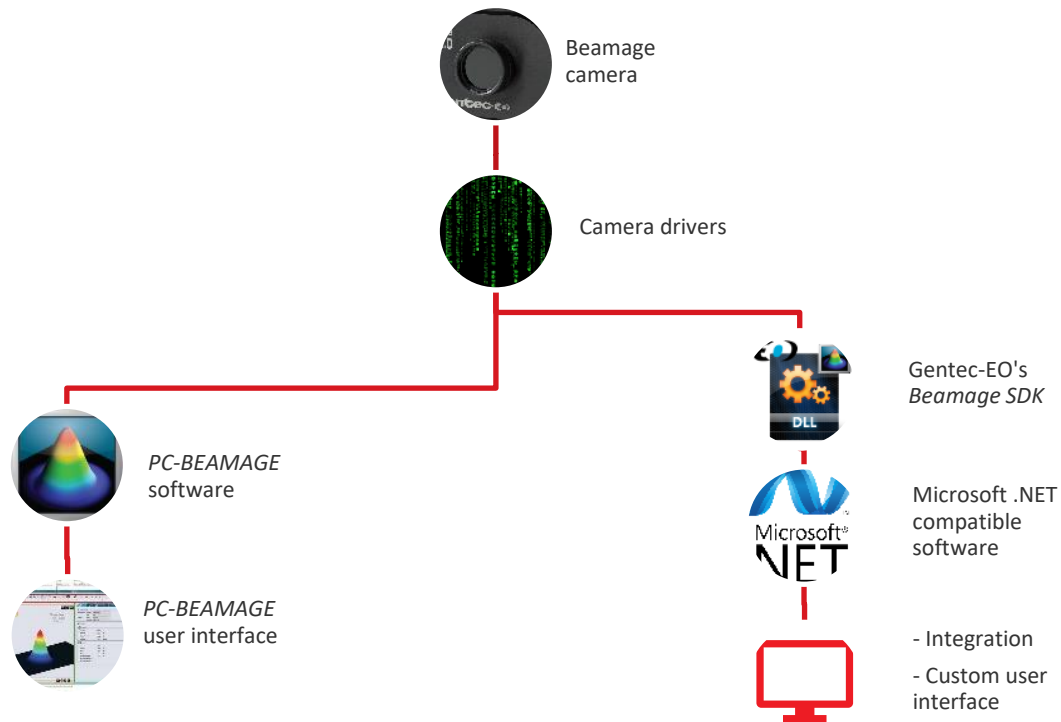


Figure 2 – A new way to use Gentec-EO's Beamage camera: the *Beamage SDK*

1.2. The *.NET compatible pipeline* for Beamage cameras

Gentec-EO's *Beamage SDK* offers basic functions such as 4 sigma and centroid calculations on beam profile. If you would like your custom Windows application to include the powerful advanced functions available in Gentec-EO's *PC-BEAMAGE* software, the *.NET compatible pipeline* will be of great use to you.

Instead of communicating directly with the camera, the *.NET compatible pipeline* option allows you to communicate with the camera via the *PC-BEAMAGE* software using a RAM pipeline. Your custom Windows software application can then control the Beamage camera and obtain nearly all measurements through the *PC-BEAMAGE* software.

Below is a simple schematic that illustrates how the *.NET compatible pipeline* works.

For more information on the *.NET compatible pipeline* and how to use it, please refer to the [PC-Beamage user manual](#).

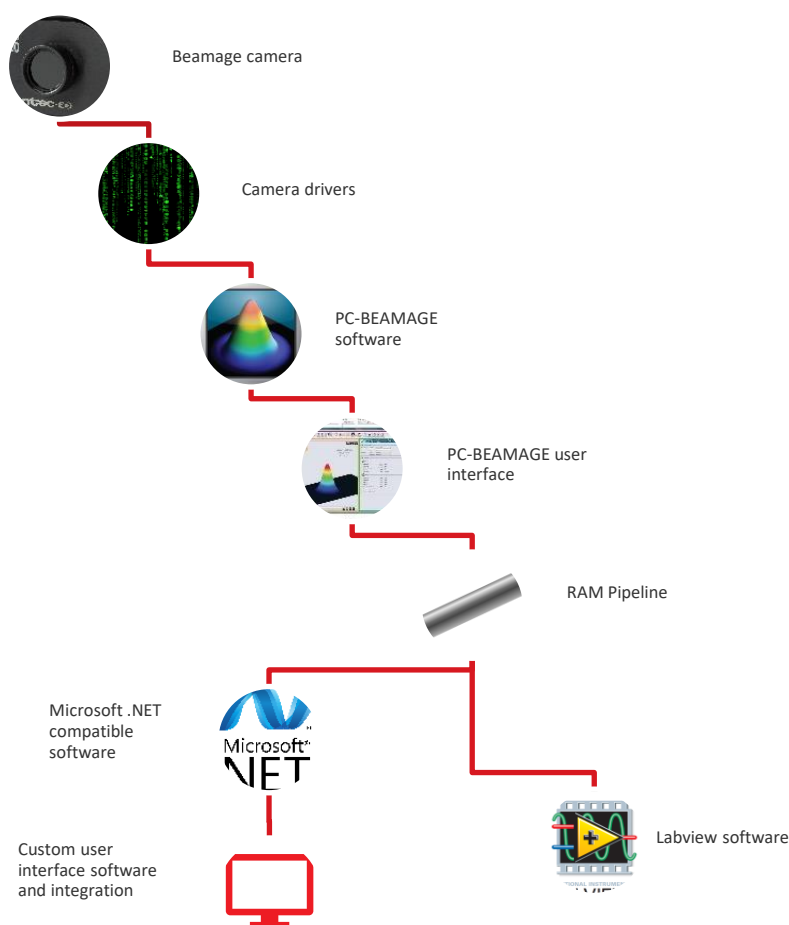


Figure 3 – Functional schematic of the .NET compatible pipeline for Beamage cameras

1.3. Getting started with the *Beamage SDK*

Before using the *Beamage SDK*, ensure that you are able to properly use the *Beamage* camera and its drivers with the *PC-BEAMAGE* software. The latest version of the *PC-BEAMAGE* software, the camera drivers and all User Manuals can be downloaded from the *Resource Center* on the following web page: <https://gentec-eo.com/products/beam-diagnostics/Beamage-3.0>.

The present document contains API documentation and code samples that will help you understand how to use each function included in the *Beamage SDK* (BSDK).

Gentec-EO also puts at your disposal a *Visual Studio* sample application: *Beamage SDK Samples*. The solution shows how to create a *Beamage* camera object, connect to it, acquire images, and how to use many other of the *Beamage SDK* functions. The present document also includes a [Quickstart Example](#) section that helps guide you get started with the *Beamage SDK Samples* solution.

1.4. What your Visual Studio solution needs

To create your first software application with *Beamage SDK*, launch *Visual Studio* and create a new project.

Next, download the *Beamage Software Development Kit* (Example-Beamage-SDK-CSharp-Vx.xx.xx.zip) from our [Download Center](#) to your computer and extract all files. Find the *BeamageSDK.dll* file and add it as a reference to your *Visual Studio* project.

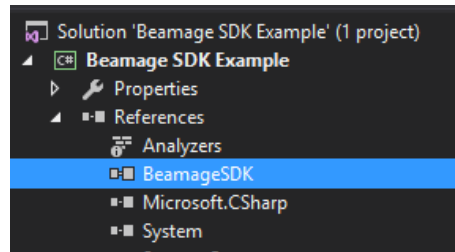


Figure 4 – *BeamageSDK.dll* has been added as a reference to the current *Visual Studio* project

That's it! Everything is now in place to start coding and communicating with the Beamage camera.

1.5. Understanding the code

Nothing beats code samples to learn to use the Beamage SDK. Below, we can see a simple class that instantiates a *BSDK* object and makes a connection with the first Beamage camera found on the computer:

Code Sample

```
using BeamageApi; // import reference from namespace BeamageApi

namespace Beamage_SDK_Example
{
    public class BeamageSdkExample
    {
        BSDK bsdk; // Beamage SDK object

        public BeamageSdkExample()
        {
            int index = 0; // Select the first camera found.

            bsdk = new BSDK(); // Create Beamage SDK

            bsdk.Detect(); // This method detects and initialize all cameras

            // Check if at least one camera is found
            if (bsdk.cameras.Count > 0)
            {
                // Get serial number
                labelSerialNumber.Text =
                    $"SerialNumber:
                    { bsdk.cameras[index].Properties.GetSerialNumber()}";

                // Get Beamage model
                labelBeamage.Text =
                    String.Format("Camera: {0}",
                    bsdk.cameras[index].Properties.Is4mSensor() ?
                    "Beamage - 4M " : "Beamage - 3.0");
            }
        }
    }
}
```


That's it! Once the object is created, all other functions in the *Beamage SDK* will be available.

For more code samples and to see how to use all functions, please refer to the [Quickstart Example](#) section. You may also consult the Table of Contents to learn more about a specific aspect of the *Beamage SDK*.

1.6. Beamage SDK Class Diagram

The *Beamage SDK* has been designed to be as simple as possible, for a fast, headache-free experience. All object classes that are part of the *Beamage SDK* begin with the letter B (for Beamage).

The main class of the *Beamage SDK* is BSDK. Objects of this class have basic functions to get the version number, to initialize, to set a canvas, to detect, and to connect to a Beamage camera. The BSDK contains a BCam (camera) class and a BErrorsManager class.

The BCam (camera) class contains most functions used to control the camera. It contains the following classes: BCamImg (images), BCamSettings (settings and parameters), and BCamProperties (read-only properties).

The BErrorsManager class helps developers understand errors that can occur when using the *Beamage SDK*.

Below is the class diagram of the *Beamage SDK*:

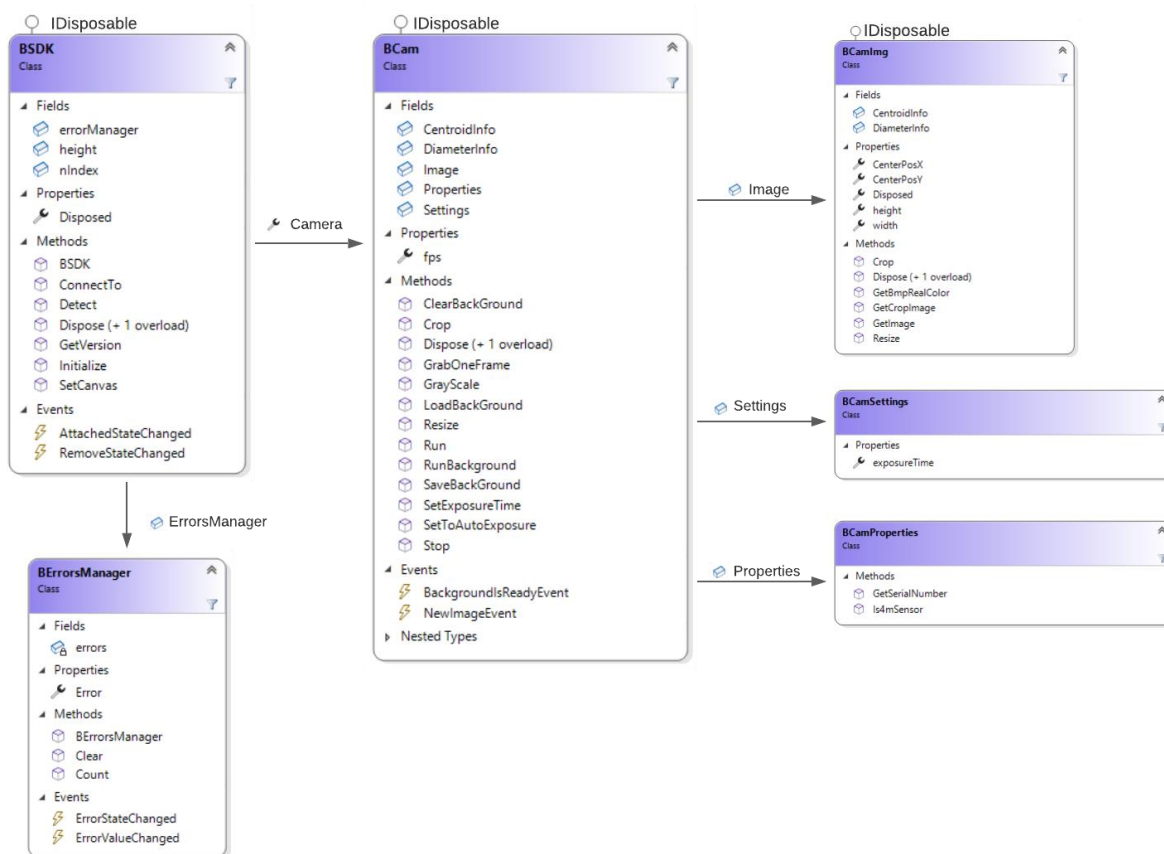


Figure 5 – Beamage SDK Class Diagram

2. Quickstart Example

A sample solution has been coded to help you get started with the *Beamage SDK*. The example is written in C# and helps you understand how to connect a camera and view images in your application.

2.1. Example: C# example with viewer

2.1.1. General information

This example has been coded for you to understand and see in action how to use the *Beamage SDK*. This *Visual Studio* solution already has the *BeamageSDK.dll* file as a reference, and can be found in our [Download Center](#). This example has a very simple user interface that was built for demonstration purposes. The code is optimized for simplicity, not performance.

Here are two screenshots of the *Beamage SDK Simple Viewer with one camera*:

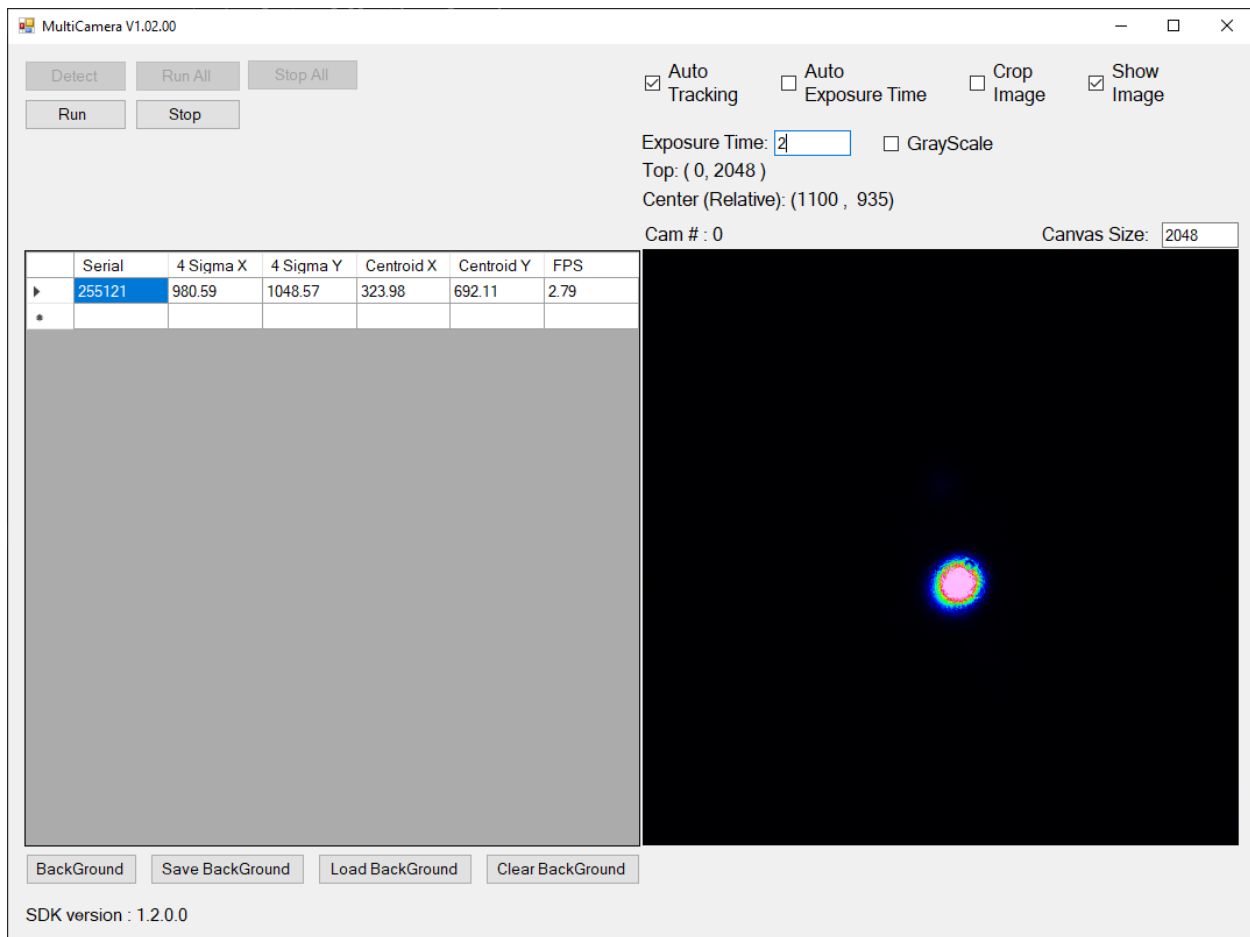


Figure 6 – *Beamage SDK Simple Viewer* user interface when a Beamage-4M is connected

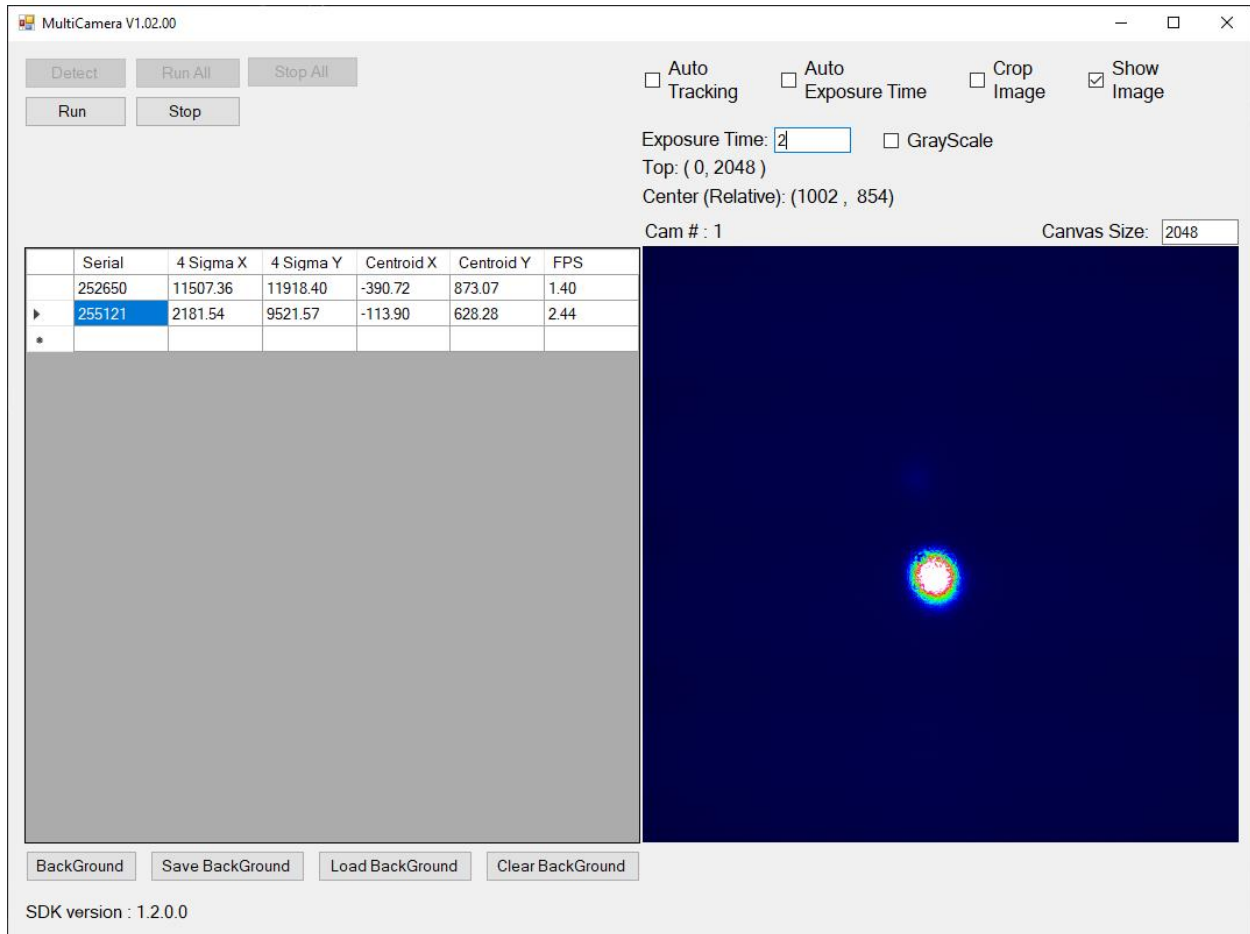


Figure 7 – Beamage SDK Simple Viewer user interface when two Beamage-4.0 are connected

The **Detect** button will open the camera drivers, detect all Beamage cameras plugged into the computer, and then initialize all cameras detected. In this example, the serial numbers of all cameras found will be displayed in the list.

Here is the source code for the **Detect** button:

Code Sample

```
private void DectectCamerasButton_Click(object sender, EventArgs e)
{
    //SDK recognizes all connected cameras.
    //If no camera is detected, a message is shown.
    try
    {
        bsdk.SetCanvas(heighthROI);
        bsdk.Detect();
    }
    catch (Exception exception)
    {
        MessageBox.Show(exception.ToString());
    }

    cameras.Clear();
    dataGridViewCameras.Rows.Clear();

    foreach (var item in bsdk.cameras)
    {
```

```
cameras.Add(item.Properties.GetSerialNumber());  
dataGridViewCameras.Rows.Add(item.Properties.GetSerialNumber(), "", "", "");  
}  
  
if (cameras.Count >= 1)  
    SetDefaultStateButtons(true);  
else  
    MessageBox.Show("No camera has been detected", "Detection Cameras",  
        MessageBoxButtons.OK, MessageBoxIcon.Warning);  
}
```

After pressing the **Detect** button, the **Run All**, **Stop All**, **Run**, **Stop**, and **Background** buttons will be enabled if at least one camera was detected.

- ▶ **Run**: Start streaming.
- ▶ **Stop**: Stop streaming.
- ▶ **Run All**: Start streaming all cameras.
- ▶ **Stop All**: Stop streaming all cameras.
- ▶ **Auto Exposure Time**: Sets the camera exposure time automatically.
- ▶ **Exposure Time**: Manually sets the camera exposure time when **Auto Exposure Time** is disabled.. Use the text box to input the value. Allowed values range from 0.6ms to 200ms
- ▶ **Auto Tracking**: Centers the beam on the canvas center.
 - Enable **Auto Exposure Time** before using this function.
- ▶ **Crop Image**: Crops the last image to center it.
- ▶ **Show Image**: Shows the last image for the selected camera.
- ▶ **Canvas Size**: Sets the height of the canvas. The default value is 2048.
- ▶ **Background**: Perform an ISO subtract of the background.
 - This function prompts the user to block the laser. Click on the **OK** button to start the background process.
 - A pop-up will appear when the background has been subtracted.
 - The Beamage must be continuously streaming during the use of this function. The fulfillment of this condition is not validated by the software. It is the user's responsibility to ensure that the camera is continuously streaming.
- ▶ **Save Background**: Saves the current background in *.csv format.
- ▶ **Load Background**: Loads a saved background.
- ▶ **Clear Background**: Clears the current background.

2.1.2. Grab images

To demonstrate how to get images from the camera, the **NewImageAllCam** thread was coded. This thread will continuously show the last image captured by the camera if **Show Image** is enabled. This thread is launched by pressing the **Run** button and stopped with the **Stop** button.

Here is the code for the **Run** button:

Code Sample

```
private void ButtonCamRun_Click(object sender, EventArgs e)
{
    //SDK initializes the selected camera and starts the NewImageAllCam Thread.
    if (selectedIndex != -1 && !selectedIndexRunList.Contains(selectedIndex))
    {
        if (connectedCamList.Contains(selectedIndex))
        {
            bsdk.cameras[selectedIndex].Run();
            bsdk.cameras[selectedIndex].Resize(heighthROI);
            bsdk.cameras[selectedIndex].SetROI(topROI, heighthROI);
        }
        else
        {
            bsdk.cameras[selectedIndex].Connect();
            bsdk.cameras[selectedIndex].Run();
            bsdk.cameras[selectedIndex].Resize(heighthROI);
            bsdk.cameras[selectedIndex].SetROI(topROI, heighthROI);
            connectedCamList.Add(selectedIndex);
        }

        bsdk.cameras[selectedIndex].NewImageEvent +=
            new EventHandler(NewImageAllCam);

        textBoxExposureTime.Text =
            bsdk.cameras[selectedIndex].Settings.exposureTime.ToString("0.000");

        selectedIndexRunList.Add(selectedIndex);
        SetStateButtonsAllCameras(false);
    }
}
```

After pressing the **Run** button, which uses the **ButtonCamRun_Click** function, the camera will begin streaming and the **NewImageAllCam** function thread will show these images in the user interface.

Here is the code for the `NewImageAllCam` function:

Code Sample

```
private void NewImageAllCam(object sender, EventArgs e)
{
    Image.GetThumbnailImageAbort myCallback =
    new Image.GetThumbnailImageAbort(ThumbnailCallback);

    //Show the SDK compute value as 4 Sigma, centroid, and FPS.
    if (dataGridViewCameras.InvokeRequired)
    {
        dataGridViewCameras.Invoke(new MethodInvoker(delegate
        {
            int nIndex = 0;
            foreach (var item in cameras)
            {
                dataGridViewCameras.Rows[nIndex].Cells["Serial Number"].Value =
                bsdk.cameras[nIndex].Properties.GetSerialNumber();
                dataGridViewCameras.Rows[nIndex].Cells["4 Sigma X"].Value =
                bsdk.cameras[nIndex].Image.diameterInfo.diameter4SigmaX.ToString("0.00");
                dataGridViewCameras.Rows[nIndex].Cells["4 Sigma Y"].Value =
                bsdk.cameras[nIndex].Image.diameterInfo.diameter4SigmaY.ToString("0.00");
                dataGridViewCameras.Rows[nIndex].Cells["Centroid X"].Value =
                bsdk.cameras[nIndex].Image.centroidInfo.centroidXPos.ToString("0.00");
                dataGridViewCameras.Rows[nIndex].Cells["Centroid Y"].Value =
                bsdk.cameras[nIndex].Image.centroidInfo.centroidYPos.ToString("0.00");
                dataGridViewCameras.Rows[nIndex].Cells["FPS"].Value =
                bsdk.cameras[nIndex].fps.ToString("0.00");
                nIndex++;
            }
        }));

        //Show the image in real color
        if ((selectedIndex >= 0) && (selectedIndex < bsdk.cameras.Count) &&
            pictureBox.InvokeRequired && bShowImage)
        {
            pictureBox.Invoke(new MethodInvoker(delegate
            {
                Image image = bsdk.cameras[selectedIndex].Image.GetBmpRealColor();
                pictureBox.Image =
                image.GetThumbnailImage(image.Width / 4, image.Height / 4, myCallback,
                IntPtr.Zero));
            }
            else
            {
                pictureBox.Image = null;
            }

            //Show the Top value
            if (labelTop.InvokeRequired)
            {
                labelTop.Invoke(new MethodInvoker(delegate { labelTop.Text =
                $"Top: ( {topROI}, {heightROI} )"; }));
            }

            //Show the Center Value
            if (labelSize.InvokeRequired)
            {
                labelSize.Invoke(new MethodInvoker(delegate { labelSize.Text
                = $"Center (Relative): ({bsdk.cameras[selectedIndex].Image.CenterPosX} ,
                { bsdk.cameras[selectedIndex].Image.CenterPosY} )"; }));
            }
        }
    }
}
```

```
    }

    //Set the exposure time value
    if (textBoxExposureTime.InvokeRequired && bExposureTime)
    {
        textBoxExposureTime.Invoke(new MethodInvoker(delegate {
            textBoxExposureTime.Text =
            bsdk.cameras[selectedIndex].Settings.exposureTime.ToString("0.000");
        }));
    }

    //Compute and set the autotraking values
    for (int i = 0; i < bsdk.cameras.Count; i++)
    {
        if (bAutotraking)
        {
            if (bsdk.cameras[i].Image.CenterPosY > heigthROI / 2) topROI +=
            Math.Abs(bsdk.cameras[i].Image.CenterPosY - heigthROI / 2) / 2;
        else
            topROI -= Math.Abs(bsdk.cameras[i].Image.CenterPosY - heigthROI / 2)
            / 2;

            // Bondaries values
            if (topROI < 0) topROI = 0;
            if (topROI > (2048 - heigthROI)) topROI = (2048 - heigthROI)
            bsdk.cameras[i].SetROI(topROI, heigthROI);
        }
    }
}
```

We can see here that the *Beamage SDK* example application simply continuously updates the `pictureBox` with a BMP image which uses the same colors as *PC-BEAMAGE*. The *Beamage SDK* also updates the values it just calculated for 4 sigma XY, centroid XY, and FPS.

2.1.3.Mathematical operations on the image buffer

When pressing the **Run** button, we've also seen that the software assigns the `bsdk.cameras[index].NewImageEvent` event to the `newImage` function. We use this function to demonstrate how to operate mathematics operations on the *Beamage SDK*.

Here is the `newImage` function code:

Code Sample

```
private void newImage(object sender, EventArgs e)
{
    int index = 0;
    // A new image has been captured by the camera
    // GetImage function returns an int array of the image
    // With the property's width and height, the image can be retrieved
    // Here, a simple average of all pixels' intensity will be shown
    var image = bsdk.cameras[index].Image.GetImage();
    int width = bsdk.cameras[index].Image.width;
    int height = bsdk.cameras[index].Image.height;

    double pixelSum = 0.0;

    for (int i = 0; i < height; i++)
    {
        for(int j = 0; j < width; j++)
        {
            pixelSum += image[i * height + j];
        }
    }

    // Make an average
    pixelSum /= (width * height);
}
```

We've seen how to perform analysis and mathematic operations directly on the camera's image buffer. Here, a simple averaging of all the pixels was performed for demonstration purposes.

2.1.4.Events

The *Beamage SDK* includes *Events*. The *Beamage SDK Samples* demonstrates how to handle them. First, assign an event handler to an event. The *EventHandler* will be called when the event is raised. Below is a code sample of two event handlers that have been assigned to functions:

Code Sample

```
// Assign an event handler for attached and removed device
bsdk.AttachedStateChanged += new EventHandler(attachedEvent);
bsdk.RemoveStateChanged += new EventHandler(removeEvent);

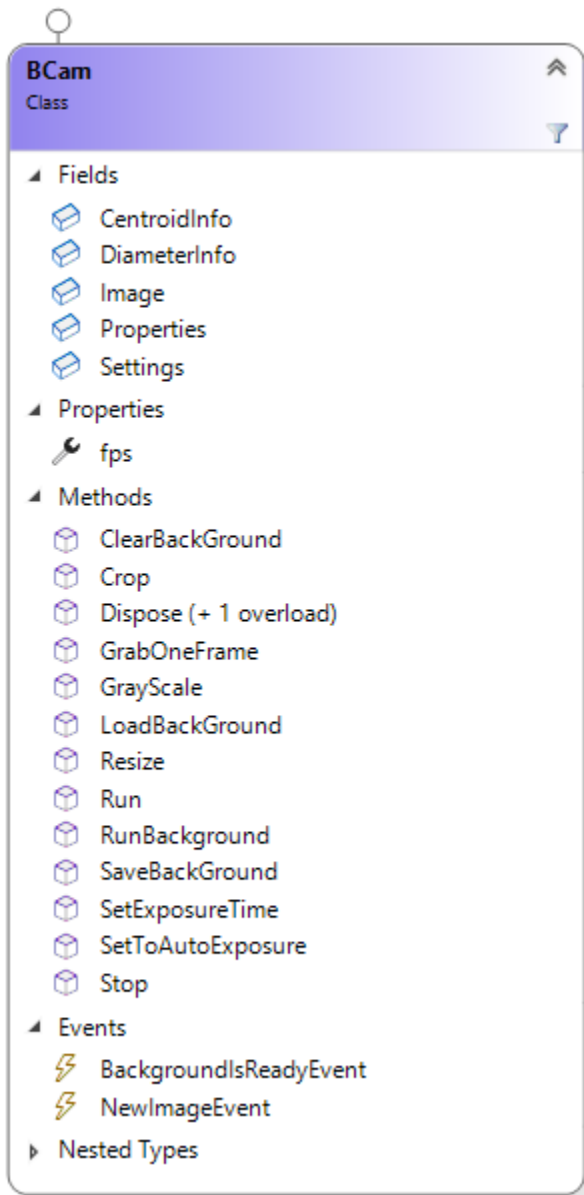
private void removeEvent(object sender, EventArgs e)
{
    // A camera has been disconnected from this PC
}

private void attachedEvent(object sender, EventArgs e)
{
    // A camera has been connected to this PC
}
```


3. Beamage API

This section provides a description of each of the classes and functions included in the *Beamage SDK*. Code samples are included.

3.1. BCam



BCam is the *Beamage SDK*'s camera class. Included are basic functions to interact with the Beamage camera. For example, the last captured image can be accessed using the **BCamImg** field.

Class declaration

```
class BCam
```

Properties

BCamCentroid *CentroidInfo*;

Get current calculated centroid XY values.

BCamDiameterInfo *DiameterInfo*;

Get current calculated diameter XY values.

For now, only the 4 sigma diameter is available.

BCamImg *Image*;

Refer to the **BCamImg** section

BCamSettings *Settings*;

Refer to the **BCamSettings** section

BCamProperties *Properties*;

Refer to the **BCamProperties** section

```
float fps { get; private set; }
```

Get the current Frames Per Second (fps) of the *Beamage* camera.

Events

event EventHandler NewImageEvent;

This event is raised every time a new image is captured by the *Beamage* camera.

Code sample

```
// Get notified when a new image is captured
bsdk.camera.NewImageEvent += new EventHandler(newImage);

private void newImage(object sender, EventArgs e)
{
    // A new image has been captured by the camera
}
```

event EventHandler BackgroundIsReadyEvent;

This event is raised when the subtract background operation is completed.

Code sample

```
bsdk.camera.BackgroundIsReadyEvent += new EventHandler(backgroundIsReady);

private void backgroundIsReady(object sender, EventArgs e)
{
    MessageBox.Show("Background is Ready");
    bsdk.camera.BackgroundIsReadyEvent -= new EventHandler(backgroundIsReady);
}
```

Functions

void Crop(bool _crop)

Set *_crop* to *True* to call the crop function *Crop(int _CenterX, int _CenterY)*. Set to *False* to deactivate.

void Dispose()

Bcam class implements *IDisposable* and requires a *Dispose* function.

void GrabOneFrame()

Grabs only one image and then stops the camera.

void GrayScale(bool _grayScale)

Set *_grayScale* to *True* to change the image to grayscale. Set to *False* to deactivate.

void RunBackground()

Performs an ISO subtract of the background. This function needs at least 10 captured frames to be effective.

void LoadBackground(string _pathFileName)

Set *_pathFileName* to the file path of the background subtraction you wish to load. Background are saved in *.csv format, see the *SaveBackground(string _pathFileName)* function for more details.

void SaveBackground(string _pathFileName)

Set *_pathFileName* to the file path where you wish to save the background subtraction. This function saves the background as a *.csv format file.

void ClearBackground()

Clear the array containing the background pixel values.

void Resize(int _size)

Set *_size* to the desired height of the canvas. The default height is 2048. The default width of 2048 cannot be modified.

void Run()

Starts capturing images from the *Beamage* camera. Runs continuously until the *StopRun()* function is called.

void StopRun()

Stop capturing images from the Beamage camera until the *Run()* function is called.

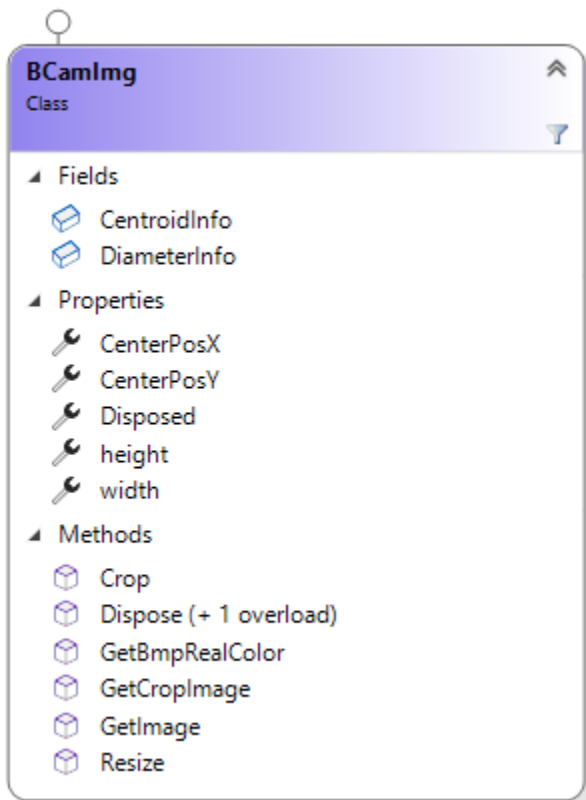
void SetExposureTime(float exposureTime)

Manually sets the exposure time of the camera. Valid values for exposure time are those between 0.06 ms and 5000 ms (5 seconds). There is no built-in input validation; respecting the aforementioned limits is the user's responsibility.

void SetToAutoExposure(bool _autoExposure)

Set *_autoExposure* to *True* to use the Beamage camera's automatic exposure time mode. Set it to *False* to deactivate it and then use the *SetExposureTime(float exposureTime)* function to control the exposure time.

3.2. BCamImg



BCamImg is a simple object class that will return a raw image buffer or a BMP image.

The first format is useful to perform mathematical operations and the second one to put on a viewer.

Class declaration

```
class BCamImg
```

Properties

```
BcamCentroid CentroidInfo;
```

The current calculated centroid XY value.

```
BcamDiameterInfo DiameterInfo;
```

The current calculated diameter XY value.

For now, only the 4 sigma diameter is available.

```
int CenterPosX
```

The X position of the center of the last captured image. Read only.

```
int CenterPosY
```

The Y position of the center of the last captured image. Read only.

```
int height
```

The height of the last captured image. Read only.

```
int width
```

The width of the last captured image. Read only.

Events

The **BCamImg** class does not contain any events.

Functions

void Crop(int _CenterX, int _CenterY)

Set *_CenterX* and *_CenterY* to crop the last image to center, this function is used to improve the calculation time, as the SDK uses fewer pixels around the beam.

Bitmap GetBmpRealColor()

This function will take the raw buffer image and create a color image (in *.bmp format) from the associated energy values, using the same colors as *PC-BEAMAGE* software . Below is the color-scale used:

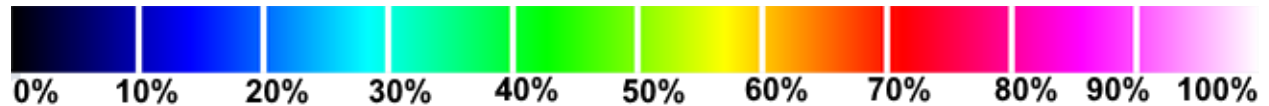


Figure 8 – SDK colors scale

Short[] GetCropImage(int _CenterX, int _CenterY, int _nWidth, int _nHeight)

Set *_Centerx*, *_CenterY*, *_nWidth*, and *_nHeight* to get the buffer data of the latest image, centered and cropped, as an array of *short*.

int[] GetImage()

Return a raw buffer of the last image. This is a one-dimension array. The elements in the array are ordered in the following way: they begin in the top left corner of the image, progress in a left-to-right and then top-to-bottom fashion and finish in the bottom right corner. This order is indicated by the red arrows below.

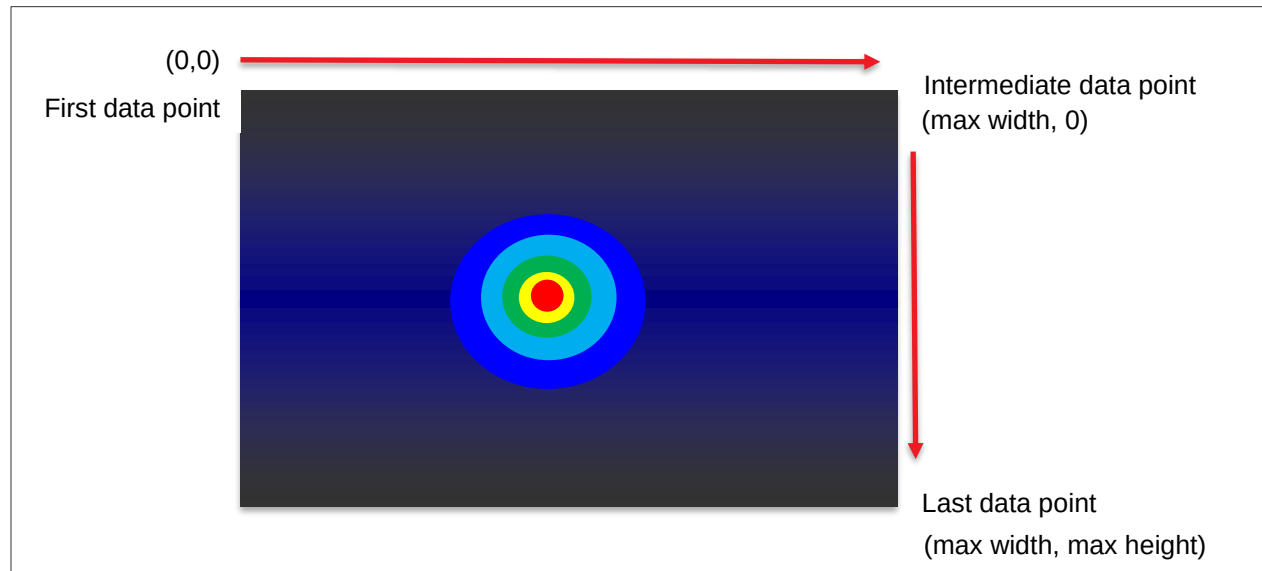


Figure 9 – Beamage image

It is simple to access any pixel of the image. Here is a code sample that shows how access all pixels to calculate their average value:

Code sample

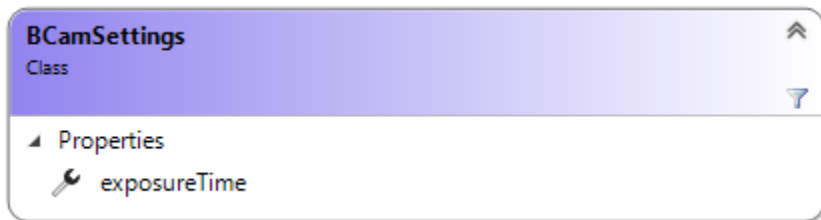
```
int index = 0; //first camera found
var image = bsdk.cameras[index].Image.GetImage();
int width = bsdk.cameras[index].Image.width;
int height = bsdk.cameras[index].Image.height;

double pixelSum = 0.0;

for (int i = 0; i < height; i++)
{
    for (int j = 0; j < width; j++)
    {
        pixelSum += image[i * height + j];
    }
}

// Make an average
pixelSum /= (width * height);
```

3.3. BCamSettings



Class declaration

```
class BCamSettings
```

Properties

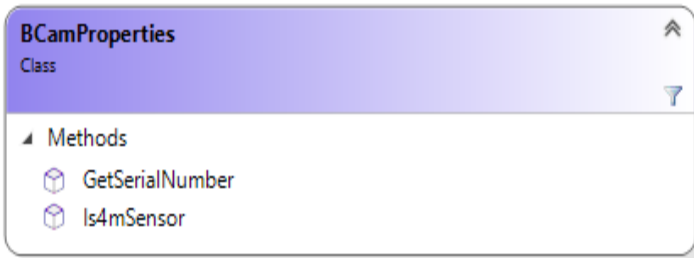
```
float exposureTime;
```

This value can be set from 0.6 ms to 200 ms.

Events

The `BCamSettings` class does not contain any events.

3.4. BCamProperties



[BCamProperties](#) are the internal properties of the Beamage camera. Contrary to the [BCamSettings](#) class, methods and properties of the [BCamProperties](#) class are read-only.

Class declaration

Properties

The [BCamProperties](#) class does not contain any properties.

Events

The [BCamProperties](#) class does not contain any events.

Functions

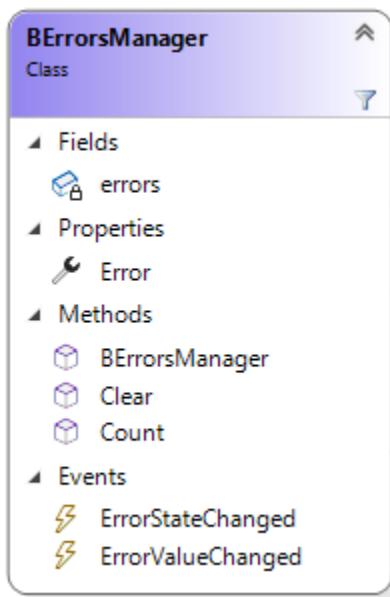
bool [Is4mSensor\(\)](#)

This function returns *True* if the Beamage camera is a Beamage-4M model and *False* if the Beamage camera is a Beamage-3.0 model.

string [GetSerialNumber\(\)](#)

Returns the Beamage camera's serial number as a string.

3.5. BErrorsManager



The **BErrorsManager** class contains events that the user can subscribe to. It will help the programmer develop applications and understand what could cause errors when using the *Beamage SDK*. To receive error messages, it is mandatory to subscribe to the **EventHandler**.

Class declaration

class BErrorsManager

Properties

string Error;

This is the error message string that gives the programmer details about errors that occur when using the code.

Events

event EventHandler ErrorStateChanged;

event EventHandler ErrorValueChanged;

This is how to subscribe to these events:

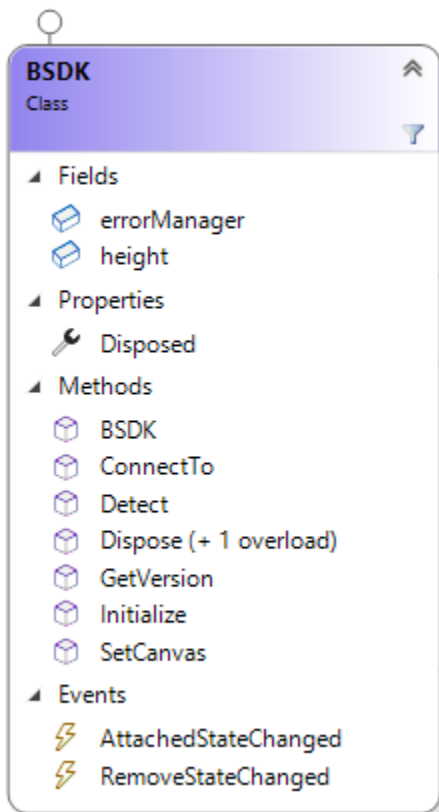
Code sample

```
// Assign an event handler for errors coming from BSDK
// The Error Manager will return errors if the SDK detects unusual behavior
bsdk.errorManager.ErrorStateChanged += new EventHandler(errorsEvent);

and

private void errorsEvent(object sender, EventArgs e)
{
    // Errors messages from the Error Manager
    string error = ((BErrorsManager)sender).Error;
    MessageBox.Show(error);
}
```

3.6. BSDK



BSDK is the main class of the *Beamage SDK*. See [the Understanding the code](#) section for more information.

Class declaration

BSDK()

Main class of objects in the *Beamage SDK*.

Properties

Events

event EventHandler AttachedStateChanged;

This *Event Handler* will raise an event each time a device is plugged into the computer. Subscribe to this event if you want to plug in a camera after starting the software or if you have multiple cameras in your setup.

event EventHandler RemoveStateChanged;

This *Event Handler* will raise an event each time a device is unplugged from the computer. It will allow you to manage some security elements or to check if your current device is still operable before performing actions.

Code sample

```
// Assign an event handler for attached and removed devices
bsdk.AttachedStateChanged += new EventHandler(attachedEvent);
bsdk.RemoveStateChanged += new EventHandler(removeEvent);
```

Functions

public void ConnectTo(int _Index)

Connect to a specific Beamage camera using its camera index. Camera index is the position in the list, so to connect the first camera the index will be 0.

public void Detect()

This function detects and initializes all Beamage cameras plugged into the computer and adds them to the list of cameras.

public string GetVersion()

This function returns the SDK version number as string.

public void Initialize()

This function calls the **Detect()** function to initialize the connected Beamage cameras.

public void SetCanvas(int _height)

Set **_height** to change the height value to the canvas, the width value is not modifiable and the default value is 2048.

void Dispose()

BSDK class implements **IDisposable** and requires a Dispose function.

LEADER IN LASER BEAM MEASUREMENT SINCE 1972



POWER & ENERGY METERS



BEAM PROFILING



THZ MEASUREMENT

CANADA

445 St-Jean-Baptiste, Suite 160
Quebec, QC, G2E 5N7, Canada

T (418) 651-8003
F (418) 651-1174

info@gentec-eo.com

UNITED STATES

5825 Jean Road Center
Lake Oswego, OR, 97035, USA

T (503) 697-1870
F (503) 697-0633

info@gentec-eo.com

JAPAN

Office No. 101, EXL111 building,
Takinogawa, Kita-ku, Tokyo
114-0023, Japan

T +81-3-5972-1290
F +81-3-5972-1291

info@gentec-eo.com

CALIBRATION CENTERS

- 445 St-Jean-Baptiste, Suite 160
Quebec, QC, G2E 5N7, Canada
- Werner von Siemens Str. 15
82140 Olching, Germany
- Office No. 101, EXL111 building,
Takinogawa, Kita-ku, Tokyo
114-0023, Japan